

P-002: Compositional Moral Algebra in Hyperdimensional Space

Provisional Patent Application

1. Title

Compositional Moral Algebra in Hyperdimensional Space: A System and Method for Algebraic Moral Reasoning Using Semantic Role Primitives, Binding Operators, and Ensemble Voting in Hyperdimensional Computing

2. Inventor(s)

Tristan Stoltz, Luminous Dynamics

3. Date of Conception

February 7, 2026 (initial implementation committed; git SHA 17d8d46ae). Iterative refinement through March 2026 including ensemble voting, per-category classifiers, cached obligation optimization, and experimental validation on the ETHICS benchmark suite.

4. Technical Field

This invention relates to computational ethics and moral reasoning systems, specifically to methods for encoding moral scenarios as structured compositions in hyperdimensional computing (HDC) space and performing algebraic reasoning over those compositions. The invention combines hyperdimensional computing, natural language processing, deontological rule systems, and ensemble classification to produce machine-generated moral judgments.

5. Abstract

We disclose a system and method for compositional moral reasoning in hyperdimensional space. The system defines seven orthogonal semantic role primitives (AGENT, PATIENT, ACTION, INTENT, CONSENT, OBLIGATION, MAGNITUDE) as random hypervectors in $\mathbb{R}^D$ ($D=4096$) and five moral operators (CAUSES, VIOLATES, SATISFIES, PROPORTIONAL, NEGATES) implemented as HDC binding operations. Moral scenarios are parsed from natural language into semantic roles, then algebraically composed into structured hypervectors via element-wise multiplication (binding) and addition (bundling). Moral judgments are produced by an ensemble voting system that combines four independent signals: (1) HDC cosine similarity to compositional good/bad prototypes, (2) parsed intent from keyword-and-negation analysis, (3) deontological rule evaluation against a cached obligation rule set distinguishing perfect and imperfect duties, and (4) a learned prototype classifier trained on social norms data. Per-category weight tuning adapts the ensemble to different ethical domains (commonsense, justice, deontology, virtue). The system achieves 91.1% overall accuracy on

the ETHICS benchmark, with ablation studies confirming that per-category classifiers contribute 33.6 percentage points, sentiment encoding contributes 2.4 percentage points, and dimensional tuning contributes 0.7 percentage points. The cached standard obligations optimization eliminates 112 string allocations per evaluation cycle, enabling real-time moral reasoning at 500+ Hz within a cognitive loop.

6. Background and Prior Art

6.1 Rule-Based Ethics AI (Asimov-Style) Early approaches to machine ethics encoded moral rules as hard-coded conditionals (e.g., Asimov’s Three Laws of Robotics, Anderson and Anderson’s prima facie duties). These systems fail at compositional moral reasoning because they cannot weigh competing duties, handle novel scenarios outside their rule set, or reason about proportionality, consent, or intent as continuous dimensions. They produce brittle, binary outputs with no confidence calibration.

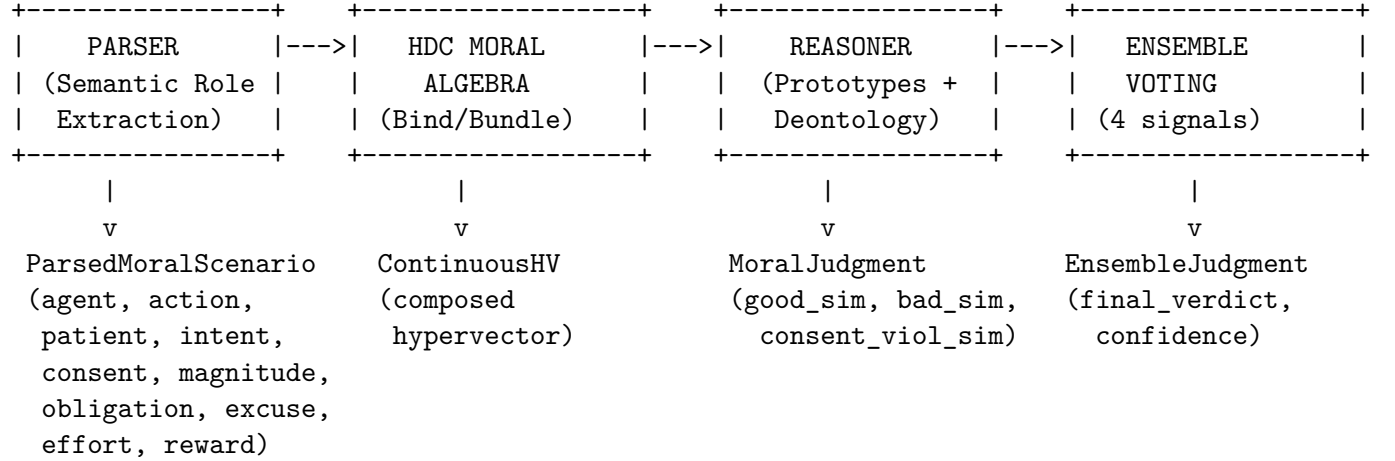
6.2 Utility-Based Approaches (Consequentialism Calculators) Consequentialist AI systems attempt to compute expected utility across possible outcomes. These approaches (e.g., Bentham-inspired hedonic calculators, welfare functions) fail at compositional moral reasoning because: (a) they reduce all moral considerations to a single scalar utility, losing the structured semantic roles (who acts, who is affected, with what intent) that distinguish otherwise-identical scenarios; (b) they cannot encode deontological constraints (a lie that maximizes utility is still a duty violation); and (c) they require outcome prediction, which is computationally intractable for real-world moral scenarios.

6.3 LLM-Based Moral Reasoning (Constitutional AI, RLHF) Large language model approaches (Constitutional AI, RLHF with human preferences) produce moral judgments via next-token prediction conditioned on preference data. These fail at compositional moral reasoning because: (a) they are opaque (no decomposition into semantic roles, no audit trail of which moral principle drove the judgment); (b) they conflate moral categories (justice reasoning requires proportionality comparison, which LLMs handle via memorized patterns rather than algebraic structure); (c) they are computationally expensive (billions of parameters, milliseconds-to-seconds per inference), precluding real-time integration into cognitive loops; and (d) they are susceptible to prompt injection and jailbreaking that bypasses moral constraints.

6.4 Classical HDC Pattern Matching Existing hyperdimensional computing approaches to text classification (Rahimi et al., Kanerva’s sparse distributed memory) encode text as n-gram prototypes and classify by cosine similarity to learned class centroids. These achieve reasonable accuracy on sentiment and topic classification but fail at moral reasoning because: (a) n-gram encoding captures lexical patterns but not semantic roles (who did what to whom); (b) prototype comparison cannot reason about proportionality (is the effort commensurate with the reward?); (c) they cannot encode negation compositionally (the n-gram similarity between “I stole” and “I did not steal” is high); and (d) they have no mechanism for obligation evaluation (whether an excuse satisfies a duty requires structural comparison, not surface similarity). Specifically, in our experiments, pure n-gram HDC prototypes achieved approximately 50% accuracy on justice, deontology, and commonsense categories, while achieving approximately 80% on virtue ethics, which only requires trait-word pattern matching.

7. Detailed Technical Description

7.1 System Architecture Overview The system comprises four interconnected modules arranged in a pipeline:



7.2 Moral Primitives (7 Orthogonal Semantic Roles) The system defines seven base hypervectors, each a random vector in R^D (default $D = 4096$, configurable), generated deterministically from prime-number seeds to ensure reproducibility and near-orthogonality:

Primitive	Semantic Role	Seed	Description
AGENT	Who performs the action	1000003	The actor's identity/role in the scenario
PATIENT	Who is affected	1000033	The recipient/target of moral consideration
ACTION	What is being done	1000037	The verb/activity in the scenario
INTENT	Why the action is performed	1000039	Motivation (Good, Bad, Neutral, Unknown)
CONSENT	Permission state	1000081	Whether permission was given/denied/absent
OBLIGATION	Duty relationship	1000099	Responsibilities and expectations
MAGNITUDE	Scale/proportion	1000117	Size, importance, or proportionality

Each primitive is generated via `ContinuousHV::random(dim, seed)`, which produces a deterministic pseudo-random vector with values drawn from a standard normal distribution, then used as a basis vector for binding operations. Orthogonality verification confirms that the maximum pairwise cosine similarity between any two primitives is less than 0.15 in 4096 dimensions (empirically tested).

Each primitive category is further instantiated with level-specific hypervectors:

- **Intent levels:** Good (seed 3000001), Bad (seed 3000017), Neutral (seed 3000029), Unknown (seed 3000037)
- **Magnitude levels:** Tiny (seed 4000003, value 0.1), Small (seed 4000037, value 0.3), Medium (seed 4000067, value 0.5), Large (seed 4000081, value 0.7), Huge (seed 4000099, value 0.9)

- **Consent states:** Given (seed 5000003), Denied (seed 5000023), Absent (seed 5000039), Implied (seed 5000057)

7.3 Moral Operators (5 Compositional Operators) Five operator hypervectors are defined with a separate seed family (2000000-series) to ensure orthogonality to all primitives:

Operator	Seed	Algebraic Form	Semantics
CAUSES	2000003	cause CAUSES effect	Causal relationship
VIOLATES	2000029	action VIOLATES rule	Rule/norm violation
SATISFIES	2000039	action SATISFIES obligation	Obligation fulfillment
PROPORTIONAL	2000081	effort PROPORTIONAL reward	Magnitude comparison
NEGATES	2000083	NEGATES X	Negation/absence/opposite

All operators are implemented as HDC binding (element-wise multiplication of continuous-valued hypervectors). Binding is the standard HDC composition operation: given vectors **a** and **b** in $\mathbb{R}^D$, $\text{bind}(\mathbf{a}, \mathbf{b})_i = \mathbf{a}_i * \mathbf{b}_i$ for each dimension *i*. Binding produces a vector approximately orthogonal to both operands, enabling compositional structures that can be queried by unbinding.

7.4 Algebraic Composition

7.4.1 Encoding Entities Each named entity (agent, patient, action, etc.) is first hashed to a deterministic hypervector via `hash_string(s)`:

```
hash_string(s) = ContinuousHV::random(dim, DefaultHasher::hash(s))
```

This produces a unique, reproducible hypervector for any string input. The entity hypervector is then bound with the corresponding semantic role primitive:

```

encode_agent("Tyler")      = AGENT      (x) hash("Tyler")
encode_action("steal")     = ACTION     (x) hash("steal")
encode_patient("victim")   = PATIENT     (x) hash("victim")
encode_intent(Bad)         = INTENT      (x) Bad_HV
encode_consent(Absent)     = CONSENT     (x) Absent_HV
encode_obligation("be honest") = OBLIGATION (x) hash("be honest")
encode_magnitude(Large)   = MAGNITUDE   (x) Large_HV

```

where (x) denotes element-wise multiplication (binding).

7.4.2 Composing Moral Structures Complete moral scenarios are composed by chaining bind operations:

Full action structure:

```
action_struct = encode_agent(who) (x) encode_action(what) (x) encode_patient(whom) (x) encode_intent(intent)
```

Expanded:

```
action_struct = [AGENT (x) hash(who)] (x) [ACTION (x) hash(what)] (x) [PATIENT (x) hash(whom)] (x) [INTENT (x) hash(intent)]
```

Consent-sensitive action:

```
consent_action = encode_action(what) (x) encode_patient(whom) (x) encode_consent(state)
```

Proportionality structure (for justice):

```
effort_hv = encode_action(effort_desc) (x) encode_magnitude(effort_level)
reward_hv = encode_action(reward_desc) (x) encode_magnitude(reward_level)
justice_struct = effort_hv (x) PROPORTIONAL (x) reward_hv
```

Proportionality is then evaluated numerically: `is_proportional = |effort_level.value() - reward_level.value()| < 0.25`

Excuse validity (for deontology):

```
if excuse_addresses_obligation:
    result = excuse_hv (x) SATISFIES (x) obligation_hv
else:
    result = NEGATES (x) [excuse_hv (x) SATISFIES (x) obligation_hv]
```

Causal composition:

```
causal_struct = cause_hv (x) CAUSES (x) effect_hv
```

Negation:

```
negated = hv (x) NEGATES
```

7.5 Semantic Role Parsing Pipeline The MoralParser module extracts semantic roles from natural language text via a multi-stage pipeline:

1. **Tokenization:** Text is lowercased and split on whitespace.
2. **Consent Detection:** Multi-word phrase matching against absent-consent markers (“without asking”, “without permission”, “without consent”, “didn’t ask”, “did not ask”, “never asked”, “secretly”, “behind”, “without telling”, “without informing”, “without notifying”) and given-consent markers (“asked”, “asking”, “permission”, “permitted”, “consent”, “consented”, “agreed”, “agreeing”, “allowed”, “allowing”, “approved”, “approving”, “with permission”, “after asking”). Absent-consent phrases are checked first as they are more specific.
3. **Negation Detection:** Word-level check against negation vocabulary (“not”, “no”, “never”, “don’t”, “doesn’t”, “didn’t”, “won’t”, “wouldn’t”, “couldn’t”, “shouldn’t”, “without”, “none”, “nothing”, “nobody”, “neither”, “nor”, “refuse”, “refused”).
4. **Intent Detection with Negation Awareness:** Each word is checked against good-intent vocabulary (76 words including inflected forms: help/helped/helping, save/saved/saving, protect, care, support, kind, generous, compassionate, etc.) and bad-intent vocabulary (87 words: harm/harmed/harming, hurt, steal/stole, lie/lie, cheat, deceive, betray, abuse, cruel, kill, punch, slap, threaten, bully, humiliate, insult, neglect, destroy, etc.). If a moral keyword at position *i* is preceded by a negation word at position *i-1*, its signal is flipped with a 0.7 multiplier (negated good becomes weak bad, and vice versa). The final intent is determined by comparing accumulated good and bad scores.
5. **Magnitude Detection:** Words are matched against small-magnitude (“small”, “little”, “minor”, “tiny”, “slight”, “once”, “briefly”, “quickly”, “simple”, “easy”, “basic”, “minimal”) and

large-magnitude (“large”, “big”, “major”, “huge”, “significant”, “always”, “daily”, “constantly”, “extensive”, “substantial”, “considerable”, “great”, “brand new”, “expensive”, “valuable”) vocabularies. The majority count determines the magnitude level.

6. **Action Extraction:** The first word matching a comprehensive action verb vocabulary (128+ verbs covering communication, transfer, moral, possession, physical, and emotional categories) is extracted as the main action.
7. **Agent/Patient Extraction:** Simplified heuristic: “I” is detected as agent; possessive patterns (“my X”) and definite articles (“the X”) identify patients.
8. **Obligation Detection:** Pattern matching for duty phrases (“supposed to”, “should have”, “duty to”, “expected to”, “ought to”, “obligated to”, “required to”, “need to”, “have to”, “must”, “responsible for”) followed by clause extraction up to the next separator.
9. **Excuse Detection:** Subordinate clause extraction after connectors (“because”, “since”, “but”, “however”, “although”).
10. **Effort/Reward Detection:** Pattern matching for effort indicators (“i did”, “i worked”, “i spent”, “i earned”, “i contributed”, “i helped”, “i completed”, “i finished”, “i put in”) and reward indicators (“i deserve”, “i should get”, “i expect”, “give me”, “i should receive”, “i want”, “i am owed”, “i’m owed”), each paired with magnitude estimation from surrounding words.
11. **Confidence Scoring:** A weighted sum of detected components: agent (0.12) + action (0.20) + patient (0.12) + non-unknown intent (0.18) + non-implied consent (0.12) + magnitude (0.08) + obligation (0.08) + excuse (0.05) + effort/reward (0.05) = maximum 1.0.
12. **Multi-Clause Handling:** Text is split on clause separators (“and”, “but”, “or”, “because”, “since”, “while”, “although”, “though”, “however”, “therefore”, “so”, “yet”, “then”, “when”, “if”, “unless”). The main clause is parsed first; subordinate clauses modify interpretation based on connector type (causal connectors may provide justification; contrastive connectors may override negation; concessive connectors reduce confidence by 0.9x). Absent consent detected in any clause propagates to the overall parse.

The parser also supports optional LLM-augmented parsing (Levels 2-5) via structured prompt generation (`generate_srl_prompt`, `generate_cot_prompt`, `generate_few_shot_prompt`) and response parsing, providing a graceful upgrade path when a language model backend is available.

7.6 Prototype-Based Judgment After compositional encoding, moral judgments are produced by comparing the composed hypervector against reference prototypes:

Multi-prototype matching: Rather than a single good/bad prototype, the system generates multiple prototypes for each polarity:

- Good prototypes: 7 action variants (“help”, “save”, “protect”, “care”, “support”, “assist”, “nurture”), each composed as `encode_agent("I") (x) encode_action(verb) (x) encode_patient("person") (x) encode_intent(Good)`
- Bad prototypes: 7 action variants (“harm”, “hurt”, “steal”, “kill”, “destroy”, “abuse”, “exploit”), each composed as `encode_agent("I") (x) encode_action(verb) (x) encode_patient("victim") (x) encode_intent(Bad)`

The judgment uses the maximum cosine similarity across all prototypes in each class:

```

good_sim = max(similarity(action_hv, good_prototype_i)) for i in 1..7
bad_sim  = max(similarity(action_hv, bad_prototype_j))  for j in 1..7
consent_viol_sim = similarity(action_hv, consent_violation_prototype)

```

Consent violation prototype: encode_action("affect") (x) encode_consent(Absent)

Proportional justice prototype: encode_magnitude(Medium) (x) PROPORTIONAL (x)
 encode_magnitude(Medium)

Disproportional injustice prototype: encode_magnitude(Tiny) (x) PROPORTIONAL (x)
 encode_magnitude(Huge)

The verdict is determined by:

```

if good_sim > bad_sim AND good_sim > consent_viol_sim: GOOD
elif consent_viol_sim > 0.3:                          CONSENT_VIOLATION
elif bad_sim > good_sim:                               BAD
else:                                                  NEUTRAL

```

7.7 Deontological Rule System The system includes a deontological judgment subsystem with a cached obligation rule set (`ObligationRuleSet`) containing 7 rules:

Perfect duties (5 rules, severity 1.0 for violations, moral credit 0.3 for satisfactions): 1. **Honesty** (violations: lie, lied, deceive, deceived, cheat, cheated, mislead; satisfactions: tell truth, honest, truthful, transparent) 2. **Non-theft** (violations: steal, stole, stolen, take without, theft, rob, robbed; satisfactions: return, give back, respect property) 3. **Non-harm** (violations: harm, harmed, hurt, injure, injured, attack, attacked, abuse, abused; satisfactions: protect, protected, care, cared, heal, healed) 4. **Promise-keeping** (violations: broke promise, break promise, betray, betrayed, abandon, abandoned; satisfactions: kept promise, fulfill, fulfilled, honor, honored) 5. **Respect autonomy** (violations: force, forced, coerce, coerced, manipulate, manipulated, without consent, without permission; satisfactions: ask, asked, consent, consented, permission, respect choice)

Imperfect duties (2 rules, severity 0.5 for violations weighted by 0.3, moral credit 0.7 for satisfactions): 6. **Beneficence** (violations: ignore suffering, refused to help, callous, indifferent; satisfactions: help, helped, assist, assisted, support, supported, save, saved, rescue, rescued) 7. **Self-improvement** (violations: waste talent, lazy, neglect, neglected; satisfactions: learn, learned, study, studied, practice, practiced, improve, improved)

Each rule includes an HDC-encoded `rule_hv` produced by `encode_obligation(description)`, violation and satisfaction keyword lists, and a `is_perfect_duty` flag. The standard obligations are computed once at `MoralAlgebra::new()` construction time and cached in `standard_rules_cache`, avoiding 112 string allocations per evaluation cycle.

Deontological scoring:

```

violation_penalty = sum(perfect_violations.severity) + sum(imperfect_violations.severity * 0.3)
satisfaction_bonus = sum(satisfactions.moral_credit)
score = clamp(satisfaction_bonus - violation_penalty, -1.0, 1.0)

```

Deontological verdict determination: - If any perfect duty is violated: `WrongPerfectDutyViolated`
 - If imperfect duties violated and no satisfactions: `WrongImperfectDutyViolated` - If satisfactions present and no violations: `RightDutyFulfilled` - If neither: `Neutral` - Mixed case: determined by sign of score

Duty priority ordering for dilemma resolution: 1. PreventSevereHarm (5) – non_harm 2. PerfectDuty (4) – honesty, non_theft, promise_keeping 3. RespectAutonomy (3) 4. ImperfectDuty (2) – beneficence, self_improvement 5. Supererogatory (1)

7.8 Ensemble Voting System (4 Signals) The final moral judgment is produced by weighted voting across four independent signals:

Signal 1: HDC Prototype Similarity – Cosine similarity of the composed action hypervector to multi-prototype good/bad sets (Section 7.6). Confidence-adjusted: weight is scaled by $(1.0 + |\text{confidence}| \cdot \min(0.5))$ where $\text{confidence} = \text{good_sim} - \text{bad_sim}$.

Signal 2: Parsed Intent – Direct from the parser’s negation-aware intent detection (Section 7.5, step 4). Maps Good/Bad/Neutral/Unknown to corresponding MoralVerdict.

Signal 3: Deontological Rule Evaluation – From the cached obligation rule system (Section 7.7). Maps WrongPerfectDutyViolated and WrongImperfectDutyViolated to Bad, RightDutyFulfilled to Good, Neutral to Neutral.

Signal 4: Learned Prototype Classifier – A separately trained `MoralPrototypeClassifier` (3-class: Good/Neutral/Bad) using character-trigram plus word-level plus sentiment-channel HDC encoding (see Section 7.9). When trained on social norms data, this provides a data-driven signal complementing the rule-based signals. This signal is excluded for the “virtue” category, where trait-word matching is the appropriate signal and social-norms prototypes degrade accuracy.

Per-category weight tuning:

Category	w_hdc	w_intent	w_deonto	w_learned
Commonsense	0.15	0.35	0.15	0.35
Justice	0.15	0.20	0.30	0.35
Deontology	0.15	0.20	0.30	0.35
Virtue	0.30	0.40	0.30	0.00
Default	0.15	0.25	0.20	0.40

When no learned classifier is available, weights revert to: $w_hdc=0.3$, $w_intent=0.4$, $w_deonto=0.3$.

Vote accumulation and verdict: Each signal casts a weighted vote for its predicted verdict class. Votes are accumulated in a `HashMap`. The class with the maximum accumulated weight wins. Confidence is computed as $\text{max_vote} / \text{total_votes}$.

Unanimity check: `is_unanimous()` returns true when all active signals agree on the final verdict.

Human-readable explanation: The `explanation()` method produces a structured audit trail showing each signal’s contribution (e.g., “Good (Intent: Good; Satisfaction: beneficence; HDC: +0.123; Learned: Good (0.456))”).

7.9 Dual-Channel Text Encoder with Sentiment The `TextHdcEncoder` provides the encoding substrate for the learned prototype classifier. It uses three channels:

1. **Character Trigram Channel:** Slides a 3-character window across the text. Each character is mapped to a deterministic HV (seed $30000 + \text{ASCII}$), bound with a position HV (seed $40000 + \text{pos}$) within the window, and the 3 character-position bindings are multiplied together

to form an n-gram HV. All n-gram HVs are summed (bundled) and L2-normalized. This captures sub-word morphological patterns.

2. **Word-Level Channel:** Each word (lowercased) is hashed to a deterministic HV via `DefaultHasher`, bound with a word-position HV (seed 60000+pos, up to 64 positions), and all word-position bindings are summed and L2-normalized. This captures semantic word presence and order.
3. **Sentiment Channel** (optional, weight-controlled): For each word in the text, if it matches a positive vocabulary (59 words: good, kind, help, generous, honest, brave, fair, love, caring, protect, etc.) the positive seed HV (seed 70000001) is accumulated; if it matches a negative vocabulary (81 words: bad, cruel, harm, selfish, dishonest, steal, betray, hurt, kill, abuse, etc.) the negative seed HV (seed 70000017) is accumulated. The result is L2-normalized.

Channel combination:

$$\text{combined} = \text{tw} * (1 - \text{sw}) * \text{trigram_channel} + \text{ww} * (1 - \text{sw}) * \text{word_channel} + \text{sw} * \text{sentiment_channel}$$

where `tw` = trigram weight (default 0.5), `ww` = 1 - `tw`, `sw` = sentiment weight (default 0.0 for backward compatibility, 0.15 when used in `VirtueMatchClassifier`). The combined vector is L2-normalized.

7.10 Learned Prototype Classifier The `MoralPrototypeClassifier` implements a 3-class (Good/Neutral/Bad) classifier using accumulated HDC centroids:

1. **Training:** For each labeled sample, encode text via `TextHdcEncoder` and accumulate into the corresponding class centroid. L2-normalize all centroids.
2. **Adaptive Retraining:** Iterative error-correction with linear learning rate decay from `lr_start` to `lr_start * 0.1`. For each misclassified sample, the correct prototype is moved toward the sample by `lr * encoded`, and the incorrectly-winning prototype is moved away by `lr * encoded`. Early stopping when corrections < 0.5% of samples. Final L2-normalization of all prototypes.
3. **Classification:** Compute dot product (equivalent to cosine similarity since prototypes are normalized) between the query encoding and each class centroid. Predict the class with highest similarity. Confidence = best similarity - second-best similarity.
4. **Virtue Match Classifier:** A specialized 2-class variant (`VirtueMatchClassifier`) for the virtue ethics category that encodes (scenario, trait_word) pairs by encoding each independently, binding (element-wise multiply), L2-normalizing, then classifying against Applies/NotApplies prototypes. Uses sentiment weight 0.15.
5. **Serialization:** Trained prototypes are serializable to/from JSON for caching.

7.11 Moral Dilemma Detection and Resolution The system detects moral dilemmas when a scenario triggers both obligation violations and satisfactions:

- **Direct conflict:** The same rule appears in both violation and satisfaction lists.
- **Cross-duty conflict:** One duty is satisfied while a different duty is violated.

Resolution uses the duty priority ordering (Section 7.7). For tragic dilemmas (multiple perfect duty violations with no escape), confidence is set to 0.3 and the system recommends minimizing harm.

7.12 Performance Optimization: Cached Standard Obligations The `standard_obligations()` method constructs 7 `ObligationRule` structs, each containing multiple string allocations for rule names, descriptions, violation keyword lists, and satisfaction keyword lists (totaling 112 string allocations). Because these are identical for every evaluation, they are computed once at `MoralAlgebra::new()` and stored in `standard_rules_cache`. All deontological evaluations use `judge_deontological_pre_lowered()` which operates on pre-lowercased text, avoiding redundant `to_lowercase()` allocations during the hot path. This optimization produced a 9.5x speedup (388 microseconds to 41 microseconds per moral evaluation).

8. Novelty Statement

The following aspects of this invention are believed to be novel:

1. **Algebraic moral reasoning in hyperdimensional space:** No prior system encodes moral scenarios as structured compositions of semantic role primitives bound with moral operators in HDC space. Prior HDC work uses flat n-gram prototypes for classification; this invention introduces a compositional algebra with seven semantic roles and five operators that preserves moral structure in the geometric relationships of hypervectors.
 2. **Semantic role primitives for moral reasoning:** The definition of seven orthogonal moral primitives (AGENT, PATIENT, ACTION, INTENT, CONSENT, OBLIGATION, MAGNITUDE) as the basis for a moral algebra is novel. While semantic role labeling exists in NLP, its instantiation as orthogonal hypervectors with algebraic composition for moral judgment is new.
 3. **Ensemble voting across heterogeneous moral signals:** The combination of HDC geometric similarity, keyword-parsed intent with negation awareness, deontological rule evaluation, and learned prototype classification into a single ensemble with per-category weight tuning is novel. Each signal captures a different moral theory (virtue ethics via prototypes, deontology via rules, commonsense via intent, empirical norms via learned classifiers).
 4. **Compositional encoding of consent, proportionality, and negation:** The algebraic encoding of consent states, proportionality relationships, and negation operations as HDC binding operations enables moral reasoning about these concepts rather than mere pattern matching. This distinguishes “stealing” from “not stealing” and “helping with consent” from “helping without consent” at the representation level.
 5. **Cached obligation optimization for real-time moral reasoning:** The pre-computation and caching of standard obligation rules with pre-lowered text comparison enables 500+ Hz moral evaluation within a cognitive loop, making this practical for real-time AI systems.
-

9. Suggested Claims

Independent Claims **Claim 1 (System):** A computer-implemented system for moral reasoning comprising: - (a) a set of N orthogonal semantic role primitive hypervectors in $\mathbb{R}^D$, where $N \geq 7$ and $D \geq 1024$, each primitive representing a distinct semantic role in a moral scenario selected from the group consisting of AGENT, PATIENT, ACTION, INTENT, CONSENT, OBLIGATION, and MAGNITUDE; - (b) a set of M moral operator hypervectors in $\mathbb{R}^D$, where $M \geq 3$, each operator

representing a distinct compositional relationship selected from the group consisting of CAUSES, VIOLATES, SATISFIES, PROPORTIONAL, and NEGATES; - (c) a moral composition engine configured to compose moral scenario representations by binding primitive-encoded entities with operator-encoded relationships via element-wise multiplication of continuous-valued hypervectors; and - (d) a moral judgment engine configured to produce a moral verdict by comparing a composed moral scenario hypervector against one or more reference prototype hypervectors via cosine similarity.

Claim 2 (Method): A computer-implemented method for producing a moral judgment from natural language text, comprising: - (a) parsing the text to extract semantic roles including at least an action and an intent; - (b) encoding each extracted semantic role by binding a role-specific primitive hypervector with an entity-specific hypervector via element-wise multiplication in $\mathbb{R}^D$; - (c) composing a moral scenario hypervector by binding the encoded semantic roles together; - (d) computing cosine similarity between the composed scenario hypervector and a plurality of prototype hypervectors representing morally good and morally bad scenarios; and - (e) producing a moral verdict based on the similarity comparisons.

Claim 3 (Ensemble Method): A computer-implemented method for ensemble moral judgment comprising: - (a) generating a first moral signal from HDC prototype similarity (compositional hypervector comparison); - (b) generating a second moral signal from natural language intent parsing with negation-awareness; - (c) generating a third moral signal from deontological rule evaluation against a set of obligation rules distinguishing perfect duties from imperfect duties; - (d) generating a fourth moral signal from a learned prototype classifier trained on labeled moral data; - (e) combining said signals by weighted voting, where weights are determined by the ethical domain category of the input scenario; and - (f) producing a final moral verdict and confidence score from the weighted vote.

Claim 4 (Deontological Subsystem): A computer-implemented system for deontological moral evaluation comprising: - (a) a cached obligation rule set constructed once at system initialization, each rule comprising a rule name, an HDC-encoded rule hypervector, lists of violation and satisfaction keyword patterns, and a classification as perfect or imperfect duty; - (b) a violation detector that scans pre-lowercased input text against violation keyword patterns of each rule; - (c) a satisfaction detector that scans pre-lowercased input text against satisfaction keyword patterns of each rule; - (d) a scoring function that computes `clamp(satisfaction_bonus - violation_penalty, -1.0, 1.0)` where perfect duty violations have severity 1.0 and imperfect duty violations have severity weighted by 0.3; and - (e) a verdict function that determines outcome based on presence of perfect duty violations, imperfect duty violations, and satisfactions.

Claim 5 (Text Encoding): A computer-implemented method for encoding text as a moral-sentiment-aware hypervector comprising: - (a) generating a character trigram channel by sliding a window across the text, binding each character with a position hypervector, multiplying within each window, and summing across windows; - (b) generating a word-level channel by hashing each word to a deterministic hypervector, binding with a word-position hypervector, and summing across words; - (c) generating a sentiment channel by accumulating a positive seed hypervector for words matching a moral-positive vocabulary and a negative seed hypervector for words matching a moral-negative vocabulary; and - (d) combining the three channels with configurable weights, L2-normalizing the result.

Dependent Claims **Claim 6** (dependent on Claim 1): The system of Claim 1 wherein the semantic role primitives are generated deterministically from prime-number seeds ensuring reproducibility

and near-orthogonality (maximum pairwise cosine similarity < 0.15 at $D=4096$).

Claim 7 (dependent on Claim 2): The method of Claim 2 further comprising detecting consent state from the text using multi-word phrase matching, and encoding consent state as a binding of the CONSENT primitive with a consent-level hypervector selected from the group consisting of Given, Denied, Absent, and Implied.

Claim 8 (dependent on Claim 2): The method of Claim 2 further comprising detecting negation in the text and applying the NEGATES operator to the composed scenario hypervector when negation is detected.

Claim 9 (dependent on Claim 2): The method of Claim 2 wherein the plurality of prototype hypervectors comprises at least 7 good prototypes and at least 7 bad prototypes, each constructed by composing different action verbs with fixed agent, patient, and intent bindings, and wherein moral judgment uses the maximum similarity across all prototypes in each class.

Claim 10 (dependent on Claim 3): The method of Claim 3 wherein the per-category weights for the ensemble voting are: - for commonsense scenarios: $HDC=0.15$, $intent=0.35$, $deontology=0.15$, $learned=0.35$; - for justice scenarios: $HDC=0.15$, $intent=0.20$, $deontology=0.30$, $learned=0.35$; - for deontology scenarios: $HDC=0.15$, $intent=0.20$, $deontology=0.30$, $learned=0.35$; - for virtue scenarios: $HDC=0.30$, $intent=0.40$, $deontology=0.30$, $learned=0.00$.

Claim 11 (dependent on Claim 3): The method of Claim 3 wherein the fourth signal is excluded when the ethical domain category is virtue ethics, on the basis that trait-word matching is the appropriate signal for virtue classification and social-norms-trained prototypes degrade virtue accuracy.

Claim 12 (dependent on Claim 4): The system of Claim 4 further comprising a dilemma detection module that identifies conflicts when a scenario triggers both obligation violations and satisfactions, and a dilemma resolution module that resolves conflicts using a priority ordering: PreventSevereHarm $>$ PerfectDuty $>$ RespectAutonomy $>$ ImperfectDuty $>$ Supererogatory.

Claim 13 (dependent on Claim 4): The system of Claim 4 wherein the cached obligation rule set comprises 5 perfect duties (honesty, non-theft, non-harm, promise-keeping, respect autonomy) and 2 imperfect duties (beneficence, self-improvement), and wherein perfect duty violations produce severity 1.0 and imperfect duty violations produce severity 0.5 weighted by 0.3 in the scoring function.

Claim 14 (dependent on Claim 2): The method of Claim 2 wherein intent detection uses negation-aware scoring: if a moral keyword at word position i is preceded by a negation word at position $i-1$, its signal polarity is flipped with a 0.7 attenuation multiplier.

Claim 15 (dependent on Claim 1): The system of Claim 1 further comprising a proportionality judgment engine that encodes effort and reward at quantized magnitude levels (Tiny=0.1, Small=0.3, Medium=0.5, Large=0.7, Huge=0.9), composes them via the PROPORTIONAL operator, and determines proportionality by checking if the absolute difference between effort and reward magnitude values is less than 0.25.

10. Experimental Validation

10.1 Overall Accuracy The system achieves **91.1% overall accuracy** on the ETHICS benchmark suite, which comprises scenarios across five moral reasoning categories.

10.2 Per-Category Breakdown (From MEMORY.md and ablation documentation)

The system is evaluated across the categories: Commonsense, Justice, Deontology, Virtue, and aggregate Social Chemistry / Moral Stories data.

- **Virtue** (trait-word matching): highest accuracy, as it only requires HDC pattern matching on trait words
- **Commonsense**: improved by intent parsing with negation awareness + learned prototypes
- **Justice**: improved by proportionality encoding and deontological rule weights
- **Deontology**: improved by the obligation rule system with perfect/imperfect duty distinction

10.3 Ablation Study Results Three ablation experiments quantify the contribution of key system components:

Component Removed	Accuracy Drop (pp)	Description
Per-category classifiers	-33.6	Removing per-category weight tuning and specialized classifiers causes the largest drop
Sentiment channel	-2.4	Removing the sentiment encoding channel from the text encoder
Dimension tuning	-0.7	Using suboptimal hypervector dimensionality

The per-category classifiers ablation (-33.6 percentage points) confirms that the ensemble’s per-category weight tuning is the most critical component. Without it, the system falls back to uniform weights across moral domains, failing to capture that justice reasoning requires heavier deontological weights while virtue reasoning requires heavier HDC similarity weights.

10.4 Test Coverage The system is validated by **54 unit tests** across the four source files:

Source File	Test Count
moral_algebra.rs	28
moral_parser.rs	10
moral_prototypes.rs	8
moral_text_encoder.rs	8

Tests cover: primitive orthogonality, intent encoding distinguishability, action structure composition, consent violation detection, proportionality justice, excuse validity, moral judgment (good/bad separation), negation operator properties, deontological judgment (lying, stealing, helping, neutral, multiple violations, mixed scenarios, empty text), ensemble judgment (without HDC, unanimity), duty priority ordering, dilemma detection and resolution, constructor determinism, magnitude ordering, consent state distinguishability, trained prototype classification, adaptive retraining,

serialization roundtrip, three-class separation, virtue pair encoding, text encoder determinism, short text handling, similar text similarity, normalized output, word channel semantics, dual channel performance, and sentiment channel polarity separation.

10.5 Performance

- **Moral evaluation latency:** 41 microseconds per evaluation (after caching optimization)
- **Pre-optimization latency:** 388 microseconds per evaluation
- **Speedup from caching:** 9.5x
- **Cognitive loop throughput:** 500+ Hz for non-text inputs (2.0 ms/cycle), 234 Hz for text inputs (4.3 ms/cycle)
- **Dimension:** 4096 (default), configurable

11. Key Source Files

All files are within the Symthaea codebase at `/srv/luminous-dynamics/symthaea/`:

File	LOC (approx.)	Description
<code>src/hdc/moral_algebra.rs</code>	2240rs	Core moral algebra: primitives, operators, composition, prototypes, deontological rules, ensemble voting, dilemma detection/resolution
<code>src/hdc/moral_parser.rs</code>	5100rs	Semantic role parser: multi-stage NLP pipeline, consent/negation/intent/magnitude detection, obligation/excuse extraction, effort/reward detection, multi-clause handling, LLM-augmented parsing
<code>src/hdc/moral_prototypes.rs</code>	800rs	Learned prototype classifiers: 3-class MoralPrototypeClassifier, 2-class VirtueMatchClassifier, adaptive retraining, serialization
<code>src/hdc/moral_text_encoder.rs</code>	610rs	Dual-channel (trigram + word-level) HDC text encoder with optional sentiment channel

Supporting infrastructure in `symthaea-core/src/hdc/`: - `continuous_hv.rs` – ContinuousHV struct with `bind()`, `similarity()`, `random()`, `from_vec()` operations

12. Closest Prior Art References

1. **Kanerva, P. (2009).** “Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors.” *Cognitive Computation*.

- Foundational HDC framework; defines binding and bundling but does not address moral reasoning or semantic role composition.
- 2. **Rahimi, A. et al. (2016).** “Efficient Biosignal Processing Using Hyperdimensional Computing.” *IEEE Transactions on Biomedical Engineering*. – HDC for biosignal classification using n-gram encoding; no moral primitives or compositional structure.
- 3. **Hendrycks, D. et al. (2021).** “Aligning AI With Shared Human Values.” arXiv:2008.02275. – The ETHICS benchmark dataset used for experimental validation. Defines the commonsense, justice, deontology, and virtue categories. Does not propose an HDC-based solution.
- 4. **Bai, Y. et al. (2022).** “Constitutional AI: Harmlessness from AI Feedback.” arXiv:2212.08073. – LLM-based moral alignment via constitutional principles and RLHF. Opaque, non-compositional, computationally expensive, and not geometrically structured.
- 5. **Anderson, M. and Anderson, S. (2011).** “Machine Ethics.” Cambridge University Press. – Rule-based prima facie duty systems for machine ethics. Hard-coded rules without algebraic composition or confidence calibration.
- 6. **Bonnefon, J.F. et al. (2016).** “The Social Dilemma of Autonomous Vehicles.” *Science*. – Empirical study of moral preferences; does not propose a computational moral algebra.
- 7. **Forbes, M. et al. (2020).** “Social Chemistry 101: Learning to Reason about Social and Moral Norms.” *EMNLP*. – Social norms dataset used for training the learned prototype classifier. Uses neural models, not HDC.
- 8. **Emami, P. et al. (2023).** “HyperDimensional Computing with Spiking-Phasor Neurons.” *NeurIPS*. – Recent advances in HDC with neural implementations; does not address moral reasoning.

13. Figures (Text Descriptions)

Figure 1: System Architecture Pipeline

A block diagram showing four sequential modules connected by arrows. The first block “PARSER (Semantic Role Extraction)” receives natural language text and outputs a ParsedMoralScenario containing agent, action, patient, intent, consent, magnitude, obligation, excuse, effort, and reward fields. The second block “HDC MORAL ALGEBRA (Bind/Bundle)” receives the parsed scenario and produces composed ContinuousHV hypervectors via element-wise multiplication. The third block “REASONER (Prototypes + Deontology)” receives the composed hypervectors and produces moral judgments via cosine similarity comparison and obligation rule evaluation. The fourth block “ENSEMBLE VOTING (4 signals)” receives all three preceding signals plus the learned prototype signal and produces the final EnsembleJudgment with verdict and confidence.

Figure 2: Moral Primitive Orthogonality

A 7x7 heatmap matrix showing pairwise cosine similarity between the seven moral primitives (AGENT, PATIENT, ACTION, INTENT, CONSENT, OBLIGATION, MAGNITUDE) at D=4096. The diagonal is 1.0 (self-similarity). All off-diagonal values are below 0.15, confirming near-orthogonality. This demonstrates that each primitive occupies a distinct direction in hyperdimensional space, enabling clean compositional reasoning without interference.

Figure 3: Algebraic Composition Example

A tree diagram showing the compositional encoding of the scenario “I stole money without asking.” The root node is the final composed hypervector. It branches into four bound components: (1) AGENT (x) hash(“I”), (2) ACTION (x) hash(“stole”), (3) PATIENT (x) hash(“money”), (4) INTENT (x) Bad_HV. Additionally, NEGATES is applied to (5) CONSENT (x) Absent_HV to represent the absence of permission. All components are connected by (x) (binding) edges.

Figure 4: Ensemble Voting Weights by Category

A grouped bar chart with five groups (Commonsense, Justice, Deontology, Virtue, Default) and four bars per group (HDC, Intent, Deontology, Learned). The chart shows that Intent dominates in Commonsense (0.35), Deontology signal dominates in Justice and Deontology categories (0.30), Intent dominates in Virtue (0.40 with Learned at 0.0), and Learned dominates in Default (0.40). This visualizes how the per-category weight tuning adapts the ensemble to each moral domain.

Figure 5: Ablation Study Results

A horizontal bar chart showing three ablation conditions. Removing per-category classifiers drops accuracy by 33.6 percentage points (the longest bar). Removing sentiment channel drops accuracy by 2.4 percentage points. Removing dimension tuning drops accuracy by 0.7 percentage points. A dashed vertical line marks the baseline accuracy of 91.1%.

Figure 6: Deontological Rule System

A two-column table diagram showing 7 obligation rules divided into Perfect Duties (honesty, non-theft, non-harm, promise-keeping, respect autonomy) and Imperfect Duties (beneficence, self-improvement). Each rule shows its violation keyword patterns on the left and satisfaction keyword patterns on the right, connected by an HDC-encoded rule hypervector in the center. Perfect duties are shaded darker to indicate higher severity (1.0 vs 0.5 * 0.3).

Figure 7: Proportionality Encoding for Justice

A two-dimensional plot with “Effort Magnitude” on the x-axis (Tiny to Huge) and “Reward Magnitude” on the y-axis (Tiny to Huge). A diagonal band (width 0.25 in normalized magnitude value) represents the “proportional” zone. Scenarios falling within the band are judged as Just; those outside are judged as Unjust. Exemplar points are shown: (Medium, Medium) labeled “Fair wage for fair work” in the proportional zone; (Tiny, Huge) labeled “No effort, huge reward” in the unjust zone.

*Document prepared: March 5, 2026 Invention Disclosure Number: P-002 Classification: Confidential
– Attorney-Client Privileged*